Optimisation SQL et serveur MySQL et Sécurité Mysql

Pourquoi optimiser les requêtes SQL ?

Imagine que tu cherches un mot dans un livre de 1000 pages :

- Sans index → tu feuilletes toutes les pages → très long
- Avec index → tu regardes l'index → tu vas directement à la page → beaucoup plus rapide

En SQL, MySQL fait pareil :

- Si la requête n'est pas optimisée, il lit chaque ligne de la table → consommation mémoire et CPU importantes
- Une bonne optimisation réduit le nombre de lignes scannées et accélère les réponses

Conseils pour optimiser une requête SQL

Éviter SELECT *

SELECT * FROM produits; *----SELECT * récupère toutes les colonnes → MySQL charge toutes les données même inutiles*

SELECT designation, prix FROM produits; *----Sélectionne seulement les colonnes nécessaires → moins de données → requête plus rapide*

Remplacer WHERE ... IN par WHERE EXISTS

Requete: SELECT * FROM commandes

WHERE client_id **IN** (SELECT id FROM clients WHERE ville='Agadir'); *----IN parcourt toutes les valeurs de la sous-requête → plus lent*

Requete: SELECT * FROM commandes c

WHERE **EXISTS** (

SELECT 1 FROM clients cl

WHERE cl.id = c.client_id AND cl.ville='Agadir'

); *----EXISTS arrête la recherche dès qu'il trouve une correspondance → plus rapide sur grandes tables*

Éviter les fonctions dans les prédicats

Requete: SELECT * FROM clients

WHERE **UPPER**(nom)='AHMED'; *----UPPER(nom) force MySQL à transformer toutes les lignes → l'index sur nom ne sera pas utilisé*

Requete: SELECT * FROM clients

WHERE nom='AHMED'; *----Comparaison directe → l'index sur nom peut être utilisé → plus rapide*

Explication :
Parfois, on veut comparer des valeurs de texte sans tenir compte des majuscules ou minuscules. Par exemple, "Ahmed", "AHMED" ou "ahmed" devraient être considérés comme identiques. C'est pourquoi certains utilisent UPPER() (ou LOWER()) pour transformer toutes les valeurs d'une colonne en majuscules (ou minuscules) avant la comparaison.

Attention :
L'utilisation d'une fonction sur une colonne dans la clause WHERE empêche MySQL d'utiliser un index sur cette colonne. Cela force MySQL à parcourir toutes les lignes de la table (full table scan), ce qui ralentit la requête sur de grandes tables.

Éviter % au début d'un LIKE

Requete: SELECT * FROM produits

WHERE designation LIKE '%chaussures'; *----% au début empêche l'utilisation d'un index → scan complet*

Requete: SELECT * FROM produits

WHERE designation LIKE 'chaussures%'; *----Index utilisable → recherche plus rapide*

Explication :
La clause LIKE permet de rechercher des motifs dans des colonnes texte. Pour indiquer une partie variable du texte, on utilise le caractère générique %.

Attention :

- *Si le % est placé **au début** du motif (ex : '%chaussures'), MySQL ne peut pas utiliser l'index sur cette colonne.*
- *Cela force MySQL à lire **toutes les lignes de la table** pour vérifier si la condition est remplie (full table scan), ce qui ralentit considérablement la requête sur de grandes tables.*

Utiliser INNER JOIN si possible

Requete: SELECT c.nom, cmd.date_commande

FROM clients c

INNER JOIN commandes cmd ON c.id = cmd.client_id; *----INNER JOIN → seulement les lignes correspondantes → plus rapide*

Requete: SELECT c.nom, cmd.date_commande

FROM clients c

LEFT JOIN commandes cmd ON c.id = cmd.client_id; *----LEFT JOIN → inclut toutes les lignes de clients même sans correspondance → plus de calculs*

Limiter DISTINCT, UNION et ORDER BY

Requete: SELECT DISTINCT ville FROM clients; *----**DISTINCT** trie et compare toutes les lignes → coûteux*

Requete: SELECT ville FROM clients ORDER BY ville; *----**ORDER BY** trie toutes les lignes → à utiliser seulement si nécessaire*

Requete: SELECT nom FROM clients

UNION

SELECT nom FROM fournisseurs; *----**UNION** combine les résultats de deux requêtes et supprime les doublons → coûteux*

Explication :

***DISTINCT** : supprime les doublons dans le résultat. MySQL doit trier et comparer toutes les lignes, ce qui peut être coûteux sur de grandes tables.*

***ORDER BY** : trie les lignes selon une ou plusieurs colonnes. Le tri peut demander beaucoup de ressources si la table est volumineuse.*

UNION *: permet de combiner les résultats de deux requêtes SELECT. Par défaut, UNION supprime les doublons, donc MySQL doit comparer toutes les lignes des deux résultats → coûteux sur de grands jeux de données.*

UNION ALL *: combine les résultats sans supprimer les doublons → plus rapide.*

Identifier le problème N+1

Requete: SELECT * FROM commandes WHERE client_id = 1; *----1 requête pour 1 client*

Requete: SELECT * FROM commandes WHERE client_id = 2; *----1 requête pour 1 client → répétitif*

Requete: SELECT * FROM commandes WHERE client_id = 3; *----1 requête pour 1 client → très lent sur beaucoup de clients*

Solution :

Requete: SELECT * FROM commandes WHERE client_id IN (1,2,3); *----1 seule requête → MySQL récupère toutes les lignes nécessaires → beaucoup plus rapide*

Requêtes préparées

Requete: PREPARE stmt1 FROM 'SELECT id, designation FROM produits WHERE id = ?'; *----Préparer la requête → MySQL analyse une seule fois*

SET @id = 5; *----Définir la valeur du paramètre*

EXECUTE stmt1 USING @id; *----Exécuter la requête préparée avec la valeur*

DEALLOCATE PREPARE stmt1; *----Libérer la requête préparée → mémoire récupérée*

Avantages :

- Performance → moins d'analyse répétée
- Sécurité → protège contre injection SQL

Explication :

Les requêtes préparées permettent de séparer la structure de la requête des valeurs à utiliser.

- *MySQL analyse et compile la requête une seule fois, même si elle est exécutée plusieurs fois avec des valeurs différentes → gain de performance.*
- *Elles protègent contre les injections SQL car les valeurs sont traitées comme des paramètres et non comme du code SQL.*

- *Utile lorsque tu exécutes plusieurs fois la même requête avec des paramètres différents.*

EXPLAIN pour analyser une requête

Requete: EXPLAIN SELECT nom, prix

FROM produits

WHERE prix > 100; *----EXPLAIN montre comment MySQL va exécuter la requête, index utilisés, type de jointure et lignes estimées*

Explication :
La commande EXPLAIN permet d'analyser une requête SQL avant de l'exécuter :

- *Elle montre quelle table sera lue et dans quel ordre.*
- *Indique si des index sont utilisés pour filtrer ou trier les données.*
- *Montre le type de jointure prévu .*
- *Estime le nombre de lignes que MySQL devra parcourir, ce qui permet de détecter les requêtes lentes.*
- *Très utile pour optimiser les requêtes et comprendre comment MySQL traite les données.*

Optimisation de la structure et du serveur

Utiliser les index

Requete: CREATE INDEX idx_client_ville ON clients(ville); *----Créer un index sur la colonne ville → MySQL peut trouver un client sans scanner toute la table*

Types d'index :

Type	Fonction
PRIMARY KEY	Unique et clusterisé → table ordonnée par cette clé
UNIQUE	Empêche les doublons
FULLTEXT	Recherche texte

Type	Fonction
SPATIAL	Données géographiques

DROP INDEX idx_client_ville ON clients; *----Supprime un index existant → MySQL ne l'utilisera plus*

Optimisation du serveur

Variables importantes

Requete: SHOW VARIABLES; *----Montre toutes les variables serveur → utile pour ajuster la configuration*

Variable	Rôle
max_connections	Nombre max d'utilisateurs simultanés
query_cache_size	Mémoire pour stocker résultats de requêtes fréquentes
innodb_buffer_pool_size	Mémoire pour stocker données et index InnoDB
tmp_table_size	Taille max des tables temporaires en mémoire

Cache

- Stocke les résultats d'une requête fréquente pour renvoyer rapidement sans recalcul
- Analogie : garder tes documents les plus utilisés sur le bureau plutôt que de fouiller dans un tiroir à chaque fois
- Exemple : page web qui affiche les 10 derniers produits → cache permet de ne pas recalculer la requête à chaque visite

Buffer

- Stocke temporairement données et index pour réduire l'accès disque
- Plus de RAM → moins de lecture sur disque → meilleure performance

- Analogie : mettre tes livres les plus utilisés sur une étagère proche de toi au lieu de les chercher à la bibliothèque
- Exemple : SELECT sur 1 million de lignes → MySQL stocke en mémoire les blocs les plus utilisés → pas besoin de relire sur disque

Autres conseils serveur

- Utiliser InnoDB plutôt que MyISAM → support transactions et clés étrangères
- Mettre MySQL à jour → gain performance et sécurité
- Disques SSD → accès rapide
- Outils de tuning automatique → MySQLTuner, tuning-primer

Phase Sécurité MySQL :

Assurer la sécurité MySQL est essentiel pour :

- **protéger** les données sensibles
- **éviter** les accès non autorisés
- **prévenir les attaques** (injection SQL, vols de données, modifications illégales)

Imagine MySQL comme une **maison** :

- Les *utilisateurs* sont les personnes qui ont les clés
- Les *privilèges* sont les pièces auxquelles ils ont accès
- Les *rôles* sont des groupes de permissions
- Le *mot de passe* est la serrure
- Le *pare-feu* empêche les étrangers d'entrer

Créer un utilisateur correctement

✓ Exemple :

Requete: CREATE USER 'ali'@'localhost' IDENTIFIED BY 'Password123!';

----CREATE USER : création d'un nouvel utilisateur MySQL

----'ali'@'localhost' : l'utilisateur ali ne peut se connecter que depuis la machine locale

----IDENTIFIED BY : définit le mot de passe sécurisé

pourquoi ?

- Sécurise les connexions
- Empêche la connexion depuis n'importe quelle machine
- Protège la base contre les accès extérieurs

Accorder des privilèges MINIMAUX (principe du "least privilege")

Mauvaise pratique

Requete: GRANT ALL PRIVILEGES ON *.* TO 'ali'@'localhost';

----Donne tous les droits partout → très dangereux !

✓ **Bonne pratique**

Requete: GRANT SELECT, INSERT ON boutique.clients TO 'ali'@'localhost';

----SELECT, INSERT : seulement les actions nécessaires

----boutique.clients : limite l'accès à une seule table

Explication

- Moins l'utilisateur a de permissions, moins il peut provoquer de dégâts
- Empêche la suppression involontaire ou malveillante des données
- Améliore la sécurité globale

Afficher les privilèges d'un utilisateur

Requete: SHOW GRANTS FOR 'ali'@'localhost';

----Montre les permissions actuelles

----Permet de vérifier si un utilisateur a trop de privilèges

Retirer les privilèges

Requete: REVOKE INSERT ON boutique.clients FROM 'ali'@'localhost';

----REVOKE : retire des privilèges

----Sécurise en ajustant les accès selon les besoins

Supprimer un utilisateur inutile

Requete: DROP USER 'ali'@'localhost';

----Supprime complètement l'utilisateur → évite les comptes oubliés

Analogie :
Un ancien employé qui garde la clé de l'entreprise = danger maximum.

Utiliser les rôles

Les rôles = groupes de privilèges.
Cela permet de gérer plus facilement de nombreux utilisateurs.

Création d'un rôle

CREATE ROLE 'role_consultation';

----Création d'un groupe de permissions

Donner des privilèges au rôle

GRANT SELECT ON boutique.* TO 'role_consultation';

----Tous les utilisateurs ayant ce rôle pourront lire les tables

Assigner le rôle à un utilisateur

GRANT 'role_consultation' TO 'samir'@'localhost';

----L'utilisateur samir récupère automatiquement les permissions du rôle

Changer le mot de passe d'un utilisateur

ALTER USER 'samir'@'localhost' IDENTIFIED BY 'NewPass2025!';

----ALTER USER : modifier les informations d'un utilisateur

----Nouveau mot de passe → plus de sécurité

Rappel

✓ Mot de passe complexe = majuscules + minuscules + chiffres + caractères spéciaux
✓ Ne jamais laisser un mot de passe par défaut

Empêcher les connexions dangereuses (accès réseau)

CREATE USER 'yassine'@'192.168.1.%' IDENTIFIED BY 'Pass2025#';

----Utilisateur autorisé uniquement depuis le réseau interne local

CREATE USER 'omar'@'%' IDENTIFIED BY 'pass';

----% signifie "toutes les machines" → fortement déconseillé !

Sécuriser la base contre l'injection SQL

✓ Toujours utiliser des requêtes préparées
✓ Ne jamais concaténer des valeurs dans du SQL
✓ Exemple :

PREPARE stmt FROM 'SELECT * FROM clients WHERE id = ?';

----MySQL ne mélange pas code et valeur → empêche l'injection

Limiter l'accès root :

- **Attention !! : root ne doit PAS se connecter à distance**

CREATE USER 'root'@'%' IDENTIFIED BY 'danger';

----Extrêmement dangereux ! → accès total depuis l'extérieur

✓ root doit uniquement être local

UPDATE mysql.user

SET Host='localhost'

WHERE User='root';

----Limiter l'accès root au serveur local → meilleure sécurité

Sécuriser MySQL avec SSL

Permet de chiffrer les connexions entre client et serveur.

SHOW VARIABLES LIKE 'have_ssl';

----Vérifie si SSL est activé

GRANT SELECT ON boutique.* TO 'soufiane'@'%' REQUIRE SSL;

----Oblige l'utilisateur à utiliser une connexion chiffrée

Sécuriser les fichiers de configuration

my.cnf (Linux) ou my.ini (Windows)

✓ Ne jamais laisser le mot de passe en clair
✓ Restreindre les permissions du fichier

Sauvegardes et restaurations (sécurité des données)

✓ Toujours protéger les fichiers de backup
✓ Les backups contiennent toutes les données sensibles

Commande :

mysqldump -u root -p boutique > sauvegarde.sql

----mysqldump : exporte la base

----Fichier à protéger → contient toutes les données

— Résumé

Élément	Objectif
CREATE USER	Créer des comptes sécurisés
GRANT / REVOKE	Contrôler les permissions
ROLES	Gérer facilement plusieurs utilisateurs
Mots de passe forts	Éviter les accès non autorisés
Pas d'accès root à distance	Haute sécurité
Requêtes préparées	Protection contre injection SQL
SSL	Connexions chiffrées
Sauvegardes sécurisées	Préserver les données